

Updated Main Sim Formula Comparison

Technical paper-to-code and implementation-equation audit for current main.py

Updated: June 8, 2026

Primary code: /Users/rhyls/Desktop/Summer26 FSC/human trust w auto sys/real sim/main.py

Helper code: /Users/rhyls/Desktop/Summer26 FSC/human trust w auto sys/real sim/simulation_helpers.py

Older notes used as starting point: MAIN_SIM_EQUATIONS_AND_CODE.md and MAIN_SIM_EULER_DYNAMICS_BEFORE_AFTER.md

Scope: Current main sim formulas: quadrotor dynamics, wind/fire spread, detection, suppression, mapping, automation, and performance score.

Technical Bottom Line

The updated code is no longer just a visual banking model. The main drone and follower drones now use stored Euler attitude states, rotor thrust mixing, first-order motor lag, torque-driven attitude dynamics, and thrust projection into world-frame acceleration.

The paper-to-code match is strongest for translational thrust projection, attitude acceleration, rotor mixing, yaw acceleration, and bounded altitude guidance. It is intentionally simplified for the motor model, yaw guidance, full Euler/body-rate mapping, and full state-space matrix forms.

The new code also adds formula-heavy wildfire-system logic: wind-biased discrete hotspot spread, range/altitude-based probabilistic detection, suppression work accumulation, mapping coverage, multi-drone effectiveness, and a four-term performance score.

Area	Current status	Main code references
Quadrotor control/dynamics	Implemented twice: reusable follower pipeline and inline lead-drone pipeline use the same equation structure.	main.py:3303-3595; main.py:8663-8963
Rotor mixing/motor lag	Current and correct. Motor lag uses $\tau_{\text{motor}} = 0.075$ s, not 0.03 s.	main.py:3199-3268
Wind/fire spread	Old spread note is stale; current interval is 1.9 s, one event per tick, 4.8-12.5 m base jumps.	main.py:447-472; main.py:8398-8535
Detection/suppression	Detection is per-frame probability with range and altitude factors; suppression is accumulated work.	main.py:8216-8375
Performance score	Four weighted terms: detection, coverage, suppression progress, and fire-out control.	main.py:4179-4350

Current Constants Snapshot

Drone constants from current main.py:159-198.

Symbol/code constant	Current value	Meaning
g	9.81 m/s ²	Gravity.
m	1.6 kg	Drone mass.
I_x, I_y, I_z	0.045, 0.052, 0.085 kg·m ²	Principal inertias used by attitude dynamics.
L_{arm}	0.28 m	Rotor arm length used for roll/pitch torques.
k_T	2.2e-5	Rotor thrust coefficient in $T_i = k_T \omega_i^2$.
k_r	0.018	Yaw moment ratio used for rotor yaw torque.
τ_{motor}	0.075 s	First-order rotor speed lag time constant.
$T_{hover, rotor}$	3.924 N	$m g / 4$.
$\omega_{hover, rotor}$	≈ 422.331 rad/s	$\sqrt{T_{hover} / k_T}$.
$u_{i, min} / u_{i, max}$	5.4936 N / 37.6704 N	0.35 m g and 2.4 m g collective thrust bounds.

Fire/detection/suppression constants from current main.py:405-446.

Symbol/code constant	Current value	Meaning
R_{det}	15.0 m	Hotspot detection radius.
λ_{det}	1.8 s ⁻¹	Base detection probability rate.
h_{min}	4.5 m	Minimum scan altitude above hotspot ground.
$T_{contain}$	30.0 s	Suppression work needed to mark a detected active hotspot contained.
$T_{extinguish}$	54.0 s	Additional suppression work after containment to mark it out.
$T_{burnout}$	180.0 s	Undetected/uncontrolled active hotspot burnout threshold.
r_{ground}	0.55 s/s	Ground firefighter suppression work rate after response delay.
r_{water}	0.9 s/s per engaged water drone	Water drone suppression work boost.
$p_{reignite}$	0.0035 s ⁻¹	Per-second reignition probability for suppressed-out hotspots.

Wind/fire spread constants from current main.py:447-472.

Symbol/code constant	Current value	Meaning
Δt_{spread}	1.9 s	Spread check interval.
N_{spread}	1 event/tick	Maximum attempted new hotspots per spread tick.
$d_{\text{min}}, d_{\text{max}}$	4.8 m, 12.5 m	Base random spread jump distance range.
s_{cell}	10.0 m	Burn-grid cell size.
$v_{\text{w,base}}$	4.0 m/s	Base wind speed.
gust speed range	± 1.8 m/s	Current wind speed is $\max(0, \text{base} + \text{gust})$.
b_{wind}	0.08 per (m/s)	Directional-bias probability factor for fire spread.
χ_{cone}	$\pm 55^\circ$	Wind-biased spread cone half-angle.
$G_{\text{down}} / G_{\text{up}}$	0.68 / 1.0	Downwind distance gain and upwind distance penalty.

Symbol Glossary

Symbol	Meaning	Code equivalent
x, y, z	World position in meters.	<code>drone.getX(), getY(), getZ()</code>
v_x, v_y, v_z	World velocity components.	<code>motion_state.velocity.x/y/z</code>
v_{des}	Desired world-frame velocity before wind blending.	<code>desired_world_vx/y/vz</code>
v_{cmd}	Commanded velocity after optional wind blending.	<code>commanded_vx/y</code>
b_w	Wind command blend coefficient.	<code>wind_command_blend</code>
e_R	Horizontal body-right unit axis.	<code>body_right</code>
e_F	Horizontal body-forward unit axis.	<code>body_forward</code>
v_R, v_F	Velocity resolved onto body-right and body-forward axes.	<code>current_body_right/forward_speed</code>
a_x, a_y, a_z	World acceleration components.	<code>world_accel_x/y/z</code>
a_R, a_F	Desired body-right/body-forward accelerations.	<code>desired_body_right/forward_accel</code>
$a_{xy,\text{max}}$	Maximum desired horizontal acceleration.	<code>DRONE_MAX_HORIZONTAL_ACCEL_METERS_PER_SECOND_SQ</code>
$a_{z,\text{max}}$	Maximum desired vertical acceleration.	<code>DRONE_MAX_VERTICAL_ACCEL_METERS_PER_SECOND_SQ</code>
ϕ	Roll angle in radians.	<code>motion_state.roll_radians</code>
θ	Pitch angle in radians.	<code>motion_state.pitch_radians</code>
ψ	Yaw/heading angle in radians.	<code>motion_state.yaw_radians</code>

ϕ_d, θ_d, ψ_d	Desired roll, pitch, and yaw commands.	desired_roll/pitch/target_yaw_radians
$\dot{\phi}, \dot{\theta}, \dot{\psi}$	Euler-angle rates.	roll_rate, pitch_rate, yaw_rate
$\ddot{\phi}, \ddot{\theta}, \ddot{\psi}$	Euler-angle accelerations.	roll_accel, pitch_accel, yaw_accel
z_{target}	Altitude target used by the hover controller.	target_altitude_meters
e_z	Altitude error $z - z_{\text{target}}$.	altitude_error
$\dot{z}_d$	Desired vertical rate.	desired_vertical_rate
k_h, k_v	Altitude guidance and vertical-rate gains.	DRONE_HOVER_GUIDANCE_GAIN / RATE_GAIN
u_1	Collective thrust.	collective_thrust_newtons
u_2 / τ_θ	Pitch torque.	pitch_torque_nm
u_3 / τ_ϕ	Roll torque.	roll_torque_nm
u_4 / τ_ψ	Yaw torque.	yaw_torque_nm
S_{fb}, S_{lr}	Intermediate front/back and left/right rotor-thrust sums.	front_back_sum / left_right_sum
T_f, T_b, T_l, T_r	Front, back, left, and right rotor thrusts.	front/back/left/right_thrust
ω_i	Rotor i angular speed.	rotor_omegas_rad_per_second[i]
k_T	Rotor thrust coefficient.	DRONE_ROTOR_THRUST_COEFFICIENT
k_r	Yaw-moment ratio.	DRONE_ROTOR_YAW_MOMENT_RATIO
τ_{motor}	Rotor first-order lag time constant.	DRONE_MOTOR_TIME_CONSTANT_SECONDS
α	Rotor low-pass blend $1 - \exp(-\Delta t / \tau_{\text{motor}})$.	blend in low_pass_value
$h(e)$	Bounded guidance function $e / (1 + e)$.	bounded_guidance(error_value)
$\text{sat}(x, a, b)$	Clamp x to interval $[a, b]$.	clamp(value, minimum, maximum)
$\Delta t / dt$	Simulation frame time step.	dt
χ_w	Wind direction heading.	wind_direction_degrees/radians
v_w	Wind speed.	wind_speed_mps
p_{bias}	Probability of drawing a spread direction from the downwind cone.	wind_direction_bias
α_{spread}	Sampled fire spread angle.	spread_angle
α_w	Wind alignment $\cos(\text{spread_angle} - \text{wind_heading})$.	wind_alignment
d_o, d	Base and wind-adjusted fire spread distance.	spread_distance
R_{det}	Hotspot detection radius.	FIRE_HOTSPOT_DETECTION_RADIUS_METER S

d_h	Horizontal drone-to-hotspot distance.	horizontal_distance
h_rel	Drone altitude above hotspot ground.	altitude_above_hotspot
P_det	Per-frame detection probability.	detection_probability
f_map,i	Mapping completion fraction for hotspot i.	hotspot_mapping_fraction(hotspot)
W_i	Suppression work accumulated for hotspot i.	hotspot.suppression_work_seconds
r_supp	Current suppression-work rate.	current_suppression_rate_per_second
M1-M4	Performance sub-scores.	m1_score ... m4_score
E_role(n)	Survey or water role effectiveness multiplier.	team_role_effectiveness(role_drone_count)
N_total	Total drone count selected for the run.	team_total_drone_count

Paper Equation Comparison

This section updates the earlier paper-equation comparison using the current line locations. The paper equations are the same equation set transcribed in the older note; the verification here is against the current source code.

Paper equation	Current implementation	Match level	Technical note
Eq. (2) translational thrust projection	main.py:3487-3493 and 8874-8880	Direct core match	The same nonlinear thrust projection is used, then extra damping/limits are applied.
Eq. (8) attitude dynamics	main.py:3449-3459 and 8836-8846	Very close	Uses post-lag reconstructed torques and stored Euler rates.
Eq. (9) Euler/body-rate map	No explicit p,q,r state	Not exact	The sim uses the small-angle assumption and integrates Euler rates directly.
Eq. (10) small-angle approximation	main.py:3461-3477 and 8848-8864	Runtime assumption	The approximation is effectively built into the state update.
Eq. (12) rotor mixing	main.py:3199-3232	Structural match	Matrix is expanded manually in front/back/left/right rotor variables.
Eq. (14) DC motor model	main.py:3235-3260	Simplified	Full motor electrical model is collapsed to first-order rotor speed lag with $\tau_{\text{motor}} = 0.075$ s.
Eq. (16)/(19) hover translation channels	main.py:3353-3373 and 8741-8761	Partial/close	Uses a $\approx g$ angle logic for desired roll/pitch, with clamps and nonlinear projection.
Eq. (21) yaw acceleration	main.py:3459 and 8846	Direct core match	$\ddot{\psi} = \tau_{\psi} / I_z$ after rotor lag.
Eq. (22), (35), (38) state-space matrices	No A/B matrix blocks	Not implemented as written	Equivalent behaviors are coded procedurally, not as closed-loop matrices.
Eq. (33) bounded altitude guidance	main.py:3187-3188; 3378-3388; 8769-8779	Direct structure	$h(e) = e / (1 + e)$, then rate/acceleration clamps.
Eq. (36)/(37) yaw guidance/PD	main.py:3420-3429 and 8811-8820	Different controller	Code uses yaw-angle error plus yaw-rate damping, not the paper's yaw-rate tracking form.
Eq. (40) position-attitude relation	main.py:3353-3373; 3487-3493; 8741-8761; 8874-8880	Conceptually close	Desired attitude is generated from desired horizontal acceleration; actual acceleration comes from thrust projection.

Current Quadrotor Control And Dynamics

Commanded Velocity And Body Axes

Manual control converts keyboard input into world-frame desired velocity using camera heading. Automation computes desired velocity from patrol, hotspot, orbit, and avoidance rules. Wind is optionally blended into the command.

```
v_cmd,x = v_des,x + b_w v_w,x
v_cmd,y = v_des,y + b_w v_w,y
e_R = [cos ψ, sin ψ]
e_F = [-sin ψ, cos ψ]
v_R = v · e_R, v_F = v · e_F
```

Here b_w is the wind command blend: 0.0 in manual mode and 0.14 for automation. e_R is the drone's body-right axis and e_F is the body-forward axis in the horizontal plane.

Desired Horizontal Acceleration To Desired Attitude

```
a_R,d = sat(k_xy (v_R,d - v_R), -a_xy,max, a_xy,max)
a_F,d = sat(k_xy (v_F,d - v_F), -a_xy,max, a_xy,max)
φ_d = sat(a_R,d / g, -φ_max, φ_max)
θ_d = sat(-a_F,d / g, -θ_max, θ_max)
```

This is the small-angle hover idea: horizontal acceleration is approximately gravity times attitude angle. Positive desired right acceleration creates positive roll; positive desired forward acceleration creates negative pitch in this coordinate convention.

Altitude Guidance And Collective Thrust

```
e_z = z - z_target
h(e_z) = e_z / (1 + |e_z|)
ẏ_d = sat(-k_h h(e_z), -v_z,max, v_z,max)
a_z,d = sat(k_v (ẏ_d - ẏ), -a_z,max, a_z,max)
u1,cmd = sat(m (g + a_z,d) / max(0.3, cos θ cos φ), u1,min, u1,max)
```

The lift projection denominator compensates for tilt so altitude control does not lose all vertical authority while the drone is pitched or rolled.

Attitude PD Torques

```
τ_θ,cmd = sat(l_x [k_p θ (θ_d - θ) - k_d θ̇], -τ_θ,max, τ_θ,max)
τ_φ,cmd = sat(l_y [k_p φ (φ_d - φ) - k_d φ̇], -τ_φ,max, τ_φ,max)
e_ψ = wrap(ψ_d - ψ)
τ_ψ,cmd = sat(l_z [k_p ψ e_ψ - k_d ψ̇], -τ_ψ,max, τ_ψ,max)
```

This is PD attitude tracking. Pitch and roll track desired angles; yaw tracks a desired heading and damps yaw rate.

Rotor Mixing And Motor Lag

```

S_fb = (u1 - u4/k_r) / 2
S_lr = (u1 + u4/k_r) / 2
T_f = max(0, (S_fb + u2/L_arm) / 2)
T_b = max(0, (S_fb - u2/L_arm) / 2)
T_l = max(0, (S_lr + u3/L_arm) / 2)
T_r = max(0, (S_lr - u3/L_arm) / 2)

ω_i,d = sqrt(max(0, T_i,d) / k_T)
α = 1 - exp(-Δt / τ_motor)
ω_i ← ω_i + α(ω_i,d - ω_i)
T_i = k_T ω_i²

```

The inverse reconstruction used after lag is $u_1 = T_f + T_b + T_l + T_r$, $u_2 = L_{\text{arm}}(T_f - T_b)$, $u_3 = L_{\text{arm}}(T_l - T_r)$, and $u_4 = k_r(-T_f - T_b + T_l + T_r)$.

Attitude And Translational Dynamics

```

θ̈ = ((l_y - l_z)/l_x) φ̇ ψ̇ + τ_θ / l_x
φ̈ = ((l_z - l_x)/l_y) θ̇ ψ̇ + τ_φ / l_y
ψ̈ = τ_ψ / l_z

ẍ = (u1/m)[cos ψ sin φ + sin ψ sin θ cos φ]
ȳ = (u1/m)[sin ψ sin φ - cos ψ sin θ cos φ]
z̈ = (u1/m) cos θ cos φ - g

d_xy = d_base + d_brake β_h
β_h = sat(1 - ||v_cmd,h|| / v_h,max, 0, 1)
d_z = d_z,active + d_z,passive β_z
β_z = sat(1 - |v_cmd,z| / v_z,max, 0, 1)
ẍ ← ẍ - d_xy v_x,  ȳ ← ȳ - d_xy v_y,  z̈ ← z̈ - d_z v_z

```

The paper dynamics are present, but damping, speed caps, angle clamps, boundary clamps, and stop-settle rules are sim/gameplay stabilizers rather than paper equations.

Wind And Fire Spread

Wind State

```

 $\chi_w = (\chi_{base} + \chi_{gust}) \bmod 360^\circ$ 
 $v_w = \max(0, v_{base} + v_{gust})$ 
 $v_{w,x} = \sin(\chi_w) v_w$ 
 $v_{w,y} = \cos(\chi_w) v_w$ 

```

The gust heading and speed move toward random targets using rate limits: 18°/s for direction and 1.5 m/s per second for speed.

Discrete Hotspot Spread

Fire spread is not a continuous PDE flame-front model. It is a discrete hotspot-spawning model that runs every 1.9 sim seconds and can create at most one new hotspot per spread tick if a valid candidate is found.

```

p_bias = min(0.9, v_w · 0.08)
if U(0,1) < p_bias:  $\alpha_{spread} \sim \text{Uniform}(\chi_w - 55^\circ, \chi_w + 55^\circ)$ 
else:  $\alpha_{spread} \sim \text{Uniform}(0, 2\pi)$ 
d_o ~ Uniform(4.8 m, 12.5 m)
a_w = cos( $\alpha_{spread} - \chi_w$ )

if a_w > 0: d = d_o + a_w v_w · 0.68
if a_w < 0: d = d_o + a_w v_w · 1.0
d ← sat(d, 0.6 · 4.8, 1.8 · 12.5)
x_new = x_source + cos( $\alpha_{spread}$ )d
y_new = y_source + sin( $\alpha_{spread}$ )d

```

Spread quantity	Current value
No-wind per-hop distance	4.8-12.5 m
No-wind equivalent hop speed	2.53-6.58 m/s, or 9.09-23.68 km/h
Wind speed range from current base/gust	2.2-5.8 m/s, before max(0, ·)
Bias probability at 4.0 m/s base wind	0.32, or 32%
Bias probability range at 2.2-5.8 m/s	0.176-0.464, or 17.6%-46.4%
Strong downwind jump at 5.8 m/s	12.5 + 5.8·0.68 = 16.444 m
Strong downwind equivalent hop speed	8.65 m/s, or 31.16 km/h
Strong upwind minimum after clamp	2.88 m
Max hotspot creation rate	1 / 1.9 = 0.526 hotspots/s

Candidate hotspots are rejected if they are outside bounds, in already burned cells, on invalid ground, or fail optional tree-clearance checks. In current code tree clearance is ignored for spread because FIRE_SPREAD_IGNORE_TREE_CLEARANCE is True.

Detection, Mapping, And Suppression

Hotspot Detection Probability

```
d_h = sqrt((x_hotspot - x_drone)^2 + (y_hotspot - y_drone)^2)
h_rel = z_drone - z_hotspot_ground
range_factor = max(0, 1 - d_h/R_det)
altitude_factor = max(0.25, min(1, h_rel/(2.5h_min)))
P_det = sat( $\lambda_{det}$  · range_factor · altitude_factor ·  $\Delta t$ , 0, 0.95)
```

Detection is impossible if $d_h > 15$ m or $h_{rel} < 4.5$ m. Automation also has a forced-detection rule: if an automated survey drone comes within 1.75 m of an active hotspot, it is marked detected immediately.

For repeated frames with changing P_{det} , the cumulative probability of at least one detection over n frames is $1 - \prod(1 - P_{det,i})$, assuming independent random draws.

Mapping Coverage

```
f_map,i = min(1, mapping_progress_i / 12 s)
coverage_score = (1/N)  $\sum_i f_{map,i}$ 
```

Mapping progress increases only for unresolved detected hotspots when at least one survey drone is near the 10.5 m orbit radius, within ± 3.0 m, and above the 4.5 m scan-altitude threshold. Each qualifying survey drone adds dt seconds of map progress per frame.

Suppression Lifecycle

```
r_supp = 0
if detected and detection_age  $\geq$  8 s: r_supp += 0.55
if water drone engaged: r_supp += 0.9 · n_water_engaged
 $W_i \leftarrow W_i + r_{supp} \Delta t$ 

ACTIVE  $\rightarrow$  CONTAINED if  $W_i \geq 30$  s
CONTAINED  $\rightarrow$  OUT if  $W_i \geq 30$  s + 54 s = 84 s
ACTIVE  $\rightarrow$  BURNED if burn_age  $\geq$  180 s
OUT  $\rightarrow$  ACTIVE with probability 0.0035  $\Delta t$  per frame
```

The variable W_i is suppression work, not wall-clock time. With only ground firefighters engaged, containment takes about $30/0.55 = 54.5$ s after the response delay. With one water drone plus ground crew, the rate is 1.45 s/s and containment takes about 20.7 s of engagement.

Fire Visual Growth Scale

```

T_growth = max(1, min(180, 30 · 1.15)) = 34.5 s
t_active = burn_age + min(detection_age, 30)
p_growth = min(1, t_active/T_growth)
scale = 0.82 + (1.38 - 0.82)p_growth

if contained:
    p_contained = sat((detection_age - 30)/54, 0, 1)
    scale ← scale · (1 - 0.34p_contained)

```

This affects presentation scale, not the mathematical spread probability directly.

Automation And Patrol Formulas

Survey Sectors And Boustrophedon Waypoints

```

S = max(1, team_survey_drone_count)
sector_width = (SPAWN_X_MAX - SPAWN_X_MIN) / S
x_min,j = SPAWN_X_MIN + j · sector_width
x_max,j = x_min,j + sector_width, except final sector uses SPAWN_X_MAX

```

Inside each sector, the patrol route is a boustrophedon sweep: x positions advance by 11 m spacing, and the target y alternates between the sector's lower and upper interior margins.

Survey Orbit

```

ω_orbit = v_orbit/R_orbit = 7.5/10.5 ≈ 0.714 rad/s
phase ← (phase + ω_orbit Δt) mod 2π
x_orbit = x_hotspot + R_orbit cos(phase)
y_orbit = y_hotspot + R_orbit sin(phase)

```

The drone then flies toward the moving orbit point with speed scaled by distance/R_orbit and clamped to the automation speed limit.

Automation Target Velocity And Altitude

```

if d_target > R_arrival:
    speed_scale = min(1, d_target/R_det)
    v_des,h = unit(target - drone) · v_h,max · speed_scale
else:
    v_des,h = 0

```

```

z_des = max(22.0, z_target + 4.5, z_ground + 8.0, z_canopy + 6.0)
v_z,des = sat(0.9(z_des - z), -v_z,max, v_z,max)

```

Tree avoidance and inter-drone avoidance can add lateral velocity corrections before final speed clamping. Those corrections are engineering safety layers, not paper quadrotor equations.

Performance Matrix

Team Effectiveness

```

E_role(n) = sat(1 + 0.16 max(0, n - 1), 0.35, 1.45)
bonus = min(0.12, 0.018 max(0, N_total - 2))

```

Survey effectiveness multiplies detection and coverage terms. Water effectiveness multiplies suppression and fire-out terms. The small extra-drone bonus increases the score ceiling for larger teams.

Detection Score M1

```

δ_i = detection_time_i - ignition_time_i, if detected
δ_i = current_time - ignition_time_i, if not yet detected
s_det,i = 1 - sat(δ_i / 8 s, 0, 1)
detection_score = (1/N) Σ_i s_det,i
M1 = sat(detection_score · E_survey, 0, 1)

```

The 8 s target comes from 120 real minutes divided by 15 real-minutes-per-sim-second. The prototype later overrides suppression pacing, but not this detection target.

Coverage, Suppression, And Fire-Out Scores

```

M2 = sat(coverage_score · E_survey, 0, 1)
M3 = sat(suppression_progress_score · E_water, 0, 1)
M4 = sat(fire_out_control_score · E_water, 0, 1)

suppression progress per detected hotspot:
OUT: 1
BURNED: 0
CONTAINED: 0.5 + 0.5·sat((W_i - 30)/54, 0, 1)
ACTIVE: 0.5·sat(W_i/30, 0, 1), with ground-crew baseline max before the 0.5 factor

fire-out control per hotspot:
OUT: 1
CONTAINED: 0.65 + 0.35·sat((W_i - 30)/54, 0, 1)
ACTIVE and detected: sat(W_i/84, 0, 0.35)
undetected or burned: 0

```

Composite Score

```
raw_score = sat((0.30M1 + 0.20M2 + 0.20M3 + 0.30M4) / 1.0, 0, 1)
performance_score = sat(raw_score + bonus, 0, 1)
reward eligible if performance_score ≥ 0.72
```

M1 and M4 have the largest weights in the current code. That makes fast discovery and actually putting fires out more important than partial mapping or partial suppression progress.

Corrections Compared With The Older Notes

- Current motor lag is $\tau_{\text{motor}} = 0.075$ s. Any old mention of 0.03 s is no longer correct.
- Current fire spread interval is 1.9 s, not 1.2 s.
- Current spread events per tick is 1, not 2.
- Current base spread distance range is 4.8-12.5 m, not 10-22 m.
- Current wind directional-bias factor is 0.08 per m/s, not 0.11.
- Current downwind distance gain is 0.68 and upwind penalty is 1.0.
- The old main.py line references around 34xx-35xx are incomplete because the lead-drone update now lives around 8640-8960 and the reusable follower function around 3303-3595.
- The simulator should be described as paper-inspired with direct matches in key equations, not as a full line-by-line implementation of the paper model.

Concise Defense Statement

The current simulator is technically defensible as a low-to-mid-fidelity operational demo. It uses real Euler-angle attitude states, torque-to-rotor mixing, first-order actuator lag, thrust-projected motion, wind-biased discrete fire spread, probabilistic detection, explicit suppression work, and a transparent weighted performance score. Its main limitations are equally explicit: it does not implement the full Euler/body-rate transform, full DC motor transfer function, full A/B state-space matrices, continuous wildfire physics, sensor false positives/negatives beyond probability gating, or high-fidelity aerodynamics.